Tutorial: Bảo vệ REST API bằng JWT với Spring Security 7 (OAuth2 Resource Server)

Oct 16, 2026 · @Ng Tuân

1. Tổng quan

Phần này nối hai tutorial trước: lấy Catalog API (CRUD sản phẩm, tài liệu hóa bằng springdoc) và bảng người dùng/vai trò của phần đăng ký–đăng nhập, rồi bảo vệ API bằng JWT. Client đổi username/mật khẩu lấy một access token có hạn 15 phút, sau đó gửi token trong header `Authorization: Bearer ...` ở mỗi request.

Khác với ứng dụng web dùng session và form login, REST API ở đây **stateless**: server không lưu phiên, mọi thông tin cần để phân quyền nằm trong token đã ký số.

Các kỹ thuật được áp dụng:

- Spring Security 7 OAuth2 Resource Server kiểm tra chữ ký, hạn dùng và issuer của JWT.
- Tự phát hành JWT ký bằng RSA (RS256) qua `JwtEncoder`; khóa đọc từ file PEM.
- Ánh xạ claim `roles` trong token thành authority `ROLE_USER`, `ROLE_ADMIN`.
- Phân quyền 2 lớp: theo URL trong `SecurityFilterChain` và theo phương thức bằng `@PreAuthorize`.
- Lỗi xác thực trả về `401` kèm header `WWW-Authenticate`, lỗi phân quyền trả `403`.
- Swagger UI có nút **Authorize** để dán token và thử API ngay trên trình duyệt.
- Test bằng `jwt()` của `spring-security-test` mà không cần phát hành token thật.

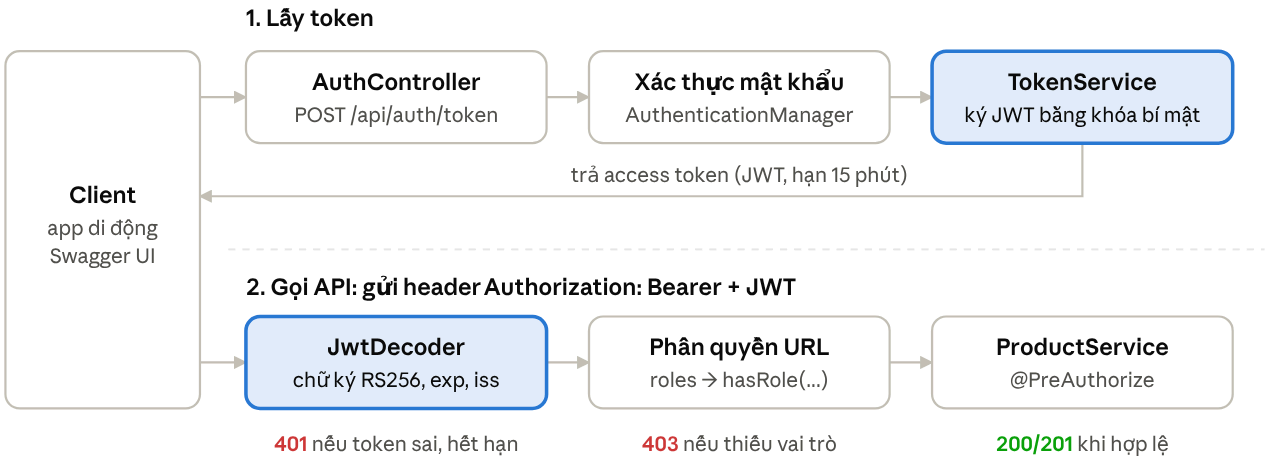
Kết quả cuối cùng:

Endpoint	Ai được gọi	Ghi chú
`POST /api/auth/token`	Mọi người	Đổi username/mật khẩu lấy access token
`GET /api/auth/me`	Có token hợp lệ	Thông tin lấy từ token đang dùng
`GET /api/v1/products/**`	Mọi người	Xem danh mục sản phẩm

Endpoint	Ai được gọi	Ghi chú
`POST` , `PUT` , `DELETE` `/api/v1/products/**`	Token có vai trò `ADMIN`	Thêm, sửa, xóa sản phẩm
`/swagger-ui.html` , `/v3/api-docs`	Mọi người	Tài liệu API

2. Luồng xác thực

Token cấp một lần, mỗi request sau tự mang token để được kiểm tra



luồng cấp token và gọi API · 2 bước

Bước 1 chạm database để kiểm tra mật khẩu; bước 2 thì không: mọi thông tin cần để phân quyền đã nằm trong token, server chỉ cần khóa công khai để kiểm tra chữ ký. Đánh đổi của cách này: token đã phát hành vẫn dùng được đến khi hết hạn, kể cả khi người dùng đã đăng xuất — bài tập ở mục 11 giải quyết vấn đề này.

3. Chuẩn bị

Dùng lại project `catalog` của tutorial tài liệu hóa REST API, rồi chép thêm phần người dùng từ tutorial đăng ký–đăng nhập.

3.1 Phiên bản

Thành phần	Phiên bản	Ghi chú
JDK	25 (LTS)	
Spring Boot	4.1.1	
Spring Security	7.1.x	Gồm `spring-security-oauth2-resource-server` và `spring-security-oauth2-jose` (Nimbus)
springdoc-openapi	3.1.1	
MySQL	8.4 LTS	Chạy qua Docker Compose
OpenSSL	Bất kỳ bản nào còn hỗ trợ	Dùng một lần để sinh cặp khóa RSA

3.2 Những gì chép sang và những gì viết mới

Nguồn	File	Việc cần làm
Tutorial REST API	Toàn bộ package `product`, `common`, `config/OpenApiConfig.java`	Giữ nguyên; mục 8 thêm `@PreAuthorize` vào `ProductService`, mục 9 sửa `OpenApiConfig`
Tutorial đăng ký–đăng nhập	`AppUser`, `Role`, `UserRepository`, `RoleRepository`, `UserService`, `UsernameAlreadyExistsException`, `DatabaseUserDetailsService`	Đổi package thành `com.fptaptech.catalog.user`
Tutorial đăng ký–đăng nhập	`db/migration/V1__init_schema.sql`	Giữ nguyên (bảng `users`, `roles`, `users_roles`)
Viết mới	`SecurityConfig`, package `auth`, `V2__products.sql`, cặp khóa RSA	Mục 5–8

Bảng `products` chuyển sang Flyway quản lý, `ddl-auto` đổi thành `validate`. `src/main/resources/db/migration/V2__products.sql`:

```
CREATE TABLE products (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(120)   NOT NULL,
    description VARCHAR(500),
    price       DECIMAL(12, 2) NOT NULL,
    created_at  DATETIME(6),
    updated_at  DATETIME(6)
);
```

3.3 Cấu trúc project

```
catalog/
├── compose.yaml
├── pom.xml
└── src/
    ├── main/
    |   ├── java/com/fptaptech/catalog/
    |   |   ├── CatalogApplication.java
    |   |   ├── config/
    |   |   |   ├── OpenApiConfig.java          # sửa: thêm security scheme
    |   |   |   └── SecurityConfig.java         # mới
    |   |   ├── auth/                           # mới
    |   |   |   ├── AuthController.java
    |   |   |   ├── LoginRequest.java
    |   |   |   ├── TokenResponse.java
    |   |   |   ├── TokenService.java
    |   |   |   └── AdminSeeder.java
    |   |   ├── common/
    |   |   |   └── GlobalExceptionHandler.java # sửa: thêm xử lý 401
    |   |   ├── product/                        # từ tutorial REST API
    |   |   └── user/                           # từ tutorial đăng ký-đăng
nhập
    |   └── resources/
    |       ├── application.yaml
    |       ├── keys/
    |       |   ├── app.key                     # khóa bí mật, KHÔNG commit
    |       |   └── app.pub
    |       └── db/migration/
    |           ├── V1__init_schema.sql
    |           └── V2__products.sql
    |
    └── test/
```

```
        ├── java/com/fptaptech/catalog/
        │   └── ApiSecurityTest.java
        └── resources/
            └── application-test.yaml
```

4. pom.xml

Starter mới cần thêm là `spring-boot-starter-security-oauth2-resource-server` (tên ở Spring Boot 4; tên cũ `spring-boot-starter-oauth2-resource-server` đã deprecated). Starter này kéo theo thư viện Nimbus JOSE dùng cho cả ký và kiểm tra JWT.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/>
    </parent>

    <groupId>com.fptaptech</groupId>
    <artifactId>catalog</artifactId>
    <version>2.0.0</version>

    <properties>
        <java.version>25</java.version>
        <springdoc.version>3.1.1</springdoc.version>
        <mapstruct.version>1.6.3</mapstruct.version>
    </properties>

    <dependencies>
        <!-- Web -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>${springdoc.version}</version>
        </dependency>

        <!-- Security + JWT -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-oauth2-resource-
server</artifactId>
        </dependency>

        <!-- Data -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-flyway</artifactId>
        </dependency>
        <dependency>
            <groupId>org.flywaydb</groupId>
            <artifactId>flyway-mysql</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-docker-compose</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
```

```
        </dependency>

        <!-- Mapping -->
        <dependency>
            <groupId>org.mapstruct</groupId>
            <artifactId>mapstruct</artifactId>
            <version>${mapstruct.version}</version>
        </dependency>

        <!-- Test -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <configuration>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.mapstruct</groupId>
                            <artifactId>mapstruct-processor</artifactId>
                            <version>${mapstruct.version}</version>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </plugin>
            <plugin>
```

```
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

5. Cặp khóa RSA và cấu hình

Token được ký bằng khóa bí mật (private key) và kiểm tra bằng khóa công khai (public key). Chỉ server phát hành token mới giữ khóa bí mật; dịch vụ nào cần kiểm tra token chỉ cần khóa công khai.

5.1 Sinh cặp khóa

Chạy ở thư mục gốc project:

```
mkdir -p src/main/resources/keys
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out
src/main/resources/keys/app.key
openssl rsa -in src/main/resources/keys/app.key -pubout -out
src/main/resources/keys/app.pub
echo "src/main/resources/keys/app.key" >> .gitignore
```

`genpkey` xuất khóa bí mật theo chuẩn PKCS#8 (`-----BEGIN PRIVATE KEY-----`), đúng định dạng Spring Security đọc được. Khóa trong `classpath` chỉ dùng cho môi trường dev; trên production trỏ tới file secret, ví dụ `file:/run/secrets/jwt-private.pem`.

5.2 application.yaml

```
spring:
  application:
    name: catalog
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate          # schema do Flyway quản lý
  mvc:
    problemdetails:
      enabled: true
```

```
app:
  jwt:
    issuer: catalog-api
    ttl: 15m                      # thời hạn access token
    public-key: classpath:keys/app.pub
    private-key: classpath:keys/app.key
  seed-admin:
    enabled: true                 # tạo tài khoản ADMIN đầu tiên (mục 8.2)
    username: admin
    password: ${ADMIN_PASSWORD:Admin#Catalog2026}

springdoc:
  api-docs:
    version: openapi_3_1
  swagger-ui:
    path: /swagger-ui.html
    persist-authorization: true   # giữ token đã dán khi tải lại Swagger UI
  paths-to-match: /api/**
```

`compose.yaml` giữ nguyên như tutorial REST API (MySQL 8.4). Mật khẩu ADMIN mặc định chỉ để chạy thử; trên server thật đặt qua biến môi trường `ADMIN_PASSWORD` và tắt `seed-admin` sau lần chạy đầu.

6. Cấu hình Spring Security

Một `SecurityFilterChain` stateless cho toàn bộ API, cùng 4 bean: `JwtDecoder` (kiểm tra token), `JwtEncoder` (phát hành token), `PasswordEncoder` và `AuthenticationManager` (kiểm tra username/mật khẩu ở endpoint cấp token).

```
package com.fptaptech.catalog.config;

import com.nimbusds.jose.jwk.JWK;
import com.nimbusds.jose.jwk.JWKSet;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.source.ImmutableJWKSet;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.ProviderManager;
import
```

```
org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import
org.springframework.security.config.annotation.method.configuration.EnableMet
hodSecurity;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.jwt.*;
import
org.springframework.security.oauth2.server.resource.authentication.JwtAuthent
icationConverter;
import
org.springframework.security.oauth2.server.resource.authentication.JwtGranted
AuthoritiesConverter;
import org.springframework.security.web.SecurityFilterChain;

import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    // Spring Security tự chuyển file PEM thành đối tượng khóa
    @Value("${app.jwt.public-key}")
    private RSAPublicKey publicKey;

    @Value("${app.jwt.private-key}")
    private RSAPrivateKey privateKey;

    @Value("${app.jwt.issuer}")
    private String issuer;

    @Bean
    SecurityFilterChain apiSecurity(HttpSecurity http) {
        http
```

```
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/swagger-ui.html", "/swagger-ui/**",
"/v3/api-docs/**", "/error").permitAll()
                .requestMatchers(HttpMethod.POST,
"/api/auth/token").permitAll()
                .requestMatchers(HttpMethod.GET,
"/api/v1/products/**").permitAll()
                .requestMatchers("/api/v1/products/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt ->
jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())));
        return http.build();
    }

    @Bean
    JwtDecoder jwtDecoder() {
        NimbusJwtDecoder decoder =
NimbusJwtDecoder.withPublicKey(publicKey).build();
        // mặc định kiểm tra exp/nbf; thêm kiểm tra issuer

decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer(issuer));
        return decoder;
    }

    @Bean
    JwtEncoder jwtEncoder() {
        JWK jwk = new
RSAKey.Builder(publicKey).privateKey(privateKey).keyID("catalog-key-
1").build();
        return new NimbusJwtEncoder(new ImmutableJWKSet<>(new JWKSet(jwk)));
    }

    private JwtAuthenticationConverter jwtAuthenticationConverter() {
        JwtGrantedAuthoritiesConverter authorities = new
JwtGrantedAuthoritiesConverter();
        authorities.setAuthoritiesClaimName("roles");
        authorities.setAuthorityPrefix(""); // claim đã chứa sẵn ROLE_USER,
ROLE_ADMIN
```

```
        JwtAuthenticationConverter converter = new
JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(authorities);
        return converter;
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

    @Bean
    AuthenticationManager authenticationManager(UserDetailsService
userDetailsService,
                                                PasswordEncoder
passwordEncoder) {
        DaoAuthenticationProvider provider = new
DaoAuthenticationProvider(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder);
        return new ProviderManager(provider);
    }
}
```

Cấu hình	Ý nghĩa
`requestMatchers(HttpMethod.GET, ...)`	Phân quyền theo cả phương thức HTTP: xem sản phẩm công khai, ghi thì cần `ADMIN`
`csrf.disable()`	An toàn cho API dùng Bearer token: trình duyệt không tự gửi header `Authorization`, nên không có nguy cơ CSRF như với cookie
`SessionCreationPolicy.STATELESS`	Không tạo `HttpSession`; mỗi request tự mang token
`oauth2ResourceServer().jwt()`	Thêm `BearerTokenAuthenticationFilter`: đọc header, giải mã, kiểm tra chữ ký RS256, `exp`, `nbf`, `iss`
`JwtGrantedAuthoritiesConverter`	Mặc định đọc claim `scope` và thêm tiền tố `SCOPE_`; ở đây đổi sang claim `roles`, không thêm tiền tố, để `hasRole("ADMIN")` hoạt động

Cấu hình	Ý nghĩa
`@EnableMethodSecurity`	Bật `@PreAuthorize` cho mục 8
`AuthenticationManager`	Khai báo tường minh vì controller cấp token (mục 7) cần gọi trực tiếp

7. Cấp token

Endpoint `POST /api/auth/token` nhận JSON username/mật khẩu, dùng `AuthenticationManager` kiểm tra với bảng `users`, rồi trả về JWT ký bằng khóa bí mật.

7.1 DTO

```
package com.fptaptech.catalog.auth;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;

public record LoginRequest(
        @Schema(example = "admin") @NotBlank String username,
        @Schema(example = "Admin#Catalog2026") @NotBlank String password
) {}
```

```
package com.fptaptech.catalog.auth;

import io.swagger.v3.oas.annotations.media.Schema;

public record TokenResponse(
        @Schema(description = "JWT dùng trong header Authorization") String
accessToken,
        @Schema(example = "Bearer") String tokenType,
        @Schema(description = "Số giây còn hiệu lực", example = "900") long
expiresIn
) {}
```

7.2 `TokenService`

Token chứa: `iss` (bên phát hành), `sub` (username), `iat` / `exp` (thời điểm phát hành/hết hạn), `jti` (mã định danh duy nhất của token) và `roles`.

```
package com.fptaptech.catalog.auth;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.oauth2.jose.jws.SignatureAlgorithm;
import org.springframework.security.oauth2.jwt.*;
import org.springframework.stereotype.Service;

import java.time.Duration;
import java.time.Instant;
import java.util.List;
import java.util.UUID;

@Service
public class TokenService {

    private final JwtEncoder encoder;
    private final String issuer;
    private final Duration ttl;

    public TokenService(JwtEncoder encoder,
                        @Value("${app.jwt.issuer}") String issuer,
                        @Value("${app.jwt.ttl}") Duration ttl) {
        this.encoder = encoder;
        this.issuer = issuer;
        this.ttl = ttl;
    }

    public TokenResponse issue(Authentication authentication) {
        Instant now = Instant.now();
        List<String> roles = authentication.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority)
                .filter(authority -> authority.startsWith("ROLE_")) // chỉ
đưa vai trò vào token
                .toList();

        JwtClaimsSet claims = JwtClaimsSet.builder()
                .issuer(issuer)
                .issuedAt(now)
                .expiresAt(now.plus(ttl))
                .subject(authentication.getName())
                .id(UUID.randomUUID().toString())
```

```
                .claim("roles", roles)
                .build();

        JwsHeader header = JwsHeader.with(SignatureAlgorithm.RS256).build();
        String token = encoder.encode(JwtEncoderParameters.from(header,
claims)).getTokenValue();
        return new TokenResponse(token, "Bearer", ttl.toSeconds());
    }
}
```

7.3 `AuthController`

```
package com.fptaptech.catalog.auth;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.ProblemDetail;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationTok
en;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.web.bind.annotation.*;

import java.time.Instant;
import java.util.List;

@RestController
@RequestMapping("/api/auth")
@Tag(name = "Auth", description = "Cấp và kiểm tra access token")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final TokenService tokenService;
```

```
    public AuthController(AuthenticationManager authenticationManager,
TokenService tokenService) {
        this.authenticationManager = authenticationManager;
        this.tokenService = tokenService;
    }

    @Operation(summary = "Đổi username/mật khẩu lấy access token")
    @ApiResponse(responseCode = "200", description = "Cấp token thành công")
    @ApiResponse(responseCode = "401", description = "Sai thông tin đăng
nhập",
            content = @Content(mediaType = "application/problem+json",
                    schema = @Schema(implementation = ProblemDetail.class)))
    @PostMapping("/token")
    public TokenResponse token(@Valid @RequestBody LoginRequest request) {
        Authentication authentication = authenticationManager.authenticate(

UsernamePasswordAuthenticationToken.unauthenticated(request.username(),
request.password()));
        return tokenService.issue(authentication);
    }

    @Operation(summary = "Thông tin của token đang dùng")
    @SecurityRequirement(name = "bearerAuth")
    @GetMapping("/me")
    public MeResponse me(@AuthenticationPrincipal Jwt jwt) {
        return new MeResponse(jwt.getSubject(),
jwt.getClaimAsStringList("roles"), jwt.getExpiresAt());
    }

    public record MeResponse(String username, List<String> roles, Instant
expiresAt) {}
}
```

7.4 Trả lỗi 401 khi sai thông tin

Thêm vào `GlobalExceptionHandler` (từ tutorial REST API). Mọi lỗi xác thực — sai mật khẩu, không có tài khoản, tài khoản bị khóa — trả cùng một thông báo để không lộ username nào đang tồn tại.

```
// import org.springframework.security.core.AuthenticationException;

@ExceptionHandler(AuthenticationException.class)
ProblemDetail handleAuthentication(AuthenticationException ex) {
    ProblemDetail pd =
ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, "Sai username hoặc
mật khẩu");
    pd.setType(URI.create(ERROR_BASE + "invalid-credentials"));
    pd.setTitle("Authentication failed");
    pd.setProperty("timestamp", Instant.now());
    return pd;
}
```

Ví dụ phản hồi thành công:

```
{
  "accessToken":
"eyJraWQiOiJjYXRhbG9nLWtleS0xIiwiYWxnIjoiUlMyNTYifQ.eyJpc3MiOi...",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```

Phần giữa của token (payload) sau khi giải mã base64:

```
{
  "iss": "catalog-api",
  "sub": "admin",
  "iat": 1792125600,
  "exp": 1792126500,
  "jti": "9d1c6a1e-4f0b-4a35-9f43-2b8f0c6d7e21",
  "roles": ["ROLE_USER", "ROLE_ADMIN"]
}
```

Payload chỉ được mã hóa base64, không được mã hóa bí mật: ai có token cũng đọc được. Không bao giờ đưa mật khẩu hay dữ liệu nhạy cảm vào claim.

8. Phân quyền theo phương thức và tài khoản ADMIN ban đầu

8.1 `@PreAuthorize` trên service

Quy tắc URL ở mục 6 chỉ chặn các request đi qua HTTP. Đặt thêm `@PreAuthorize` lên `ProductService` để thao tác ghi vẫn được bảo vệ khi service được gọi từ nơi khác (job định kỳ, import file, controller mới viết thêm mà quên cấu hình URL). Đây là nguyên tắc phòng thủ nhiều lớp.

Sửa `ProductService` từ tutorial REST API, chỉ thêm annotation:

```
import org.springframework.security.access.prepost.PreAuthorize;

@Service
@Transactional(readOnly = true)
public class ProductService {

    // ... findAll, findById giữ nguyên, không cần quyền

    @PreAuthorize("hasRole('ADMIN')")
    @Transactional
    public ProductResponse create(ProductRequest request) { /* giữ nguyên */
}

    @PreAuthorize("hasRole('ADMIN')")
    @Transactional
    public ProductResponse update(Long id, ProductRequest request) { /* giữ
nguyên */ }

    @PreAuthorize("hasRole('ADMIN')")
    @Transactional
    public void delete(Long id) { /* giữ nguyên */ }
}
```

Khi điều kiện không thỏa, Spring ném `AccessDeniedException` trước khi chạy vào thân phương thức; với request HTTP, Spring Security chuyển nó thành `403 Forbidden`.

8.2 Tạo tài khoản ADMIN đầu tiên

API không có chức năng đăng ký, nên cần một tài khoản quản trị để bắt đầu. `AdminSeeder` chạy một lần lúc khởi động, chỉ khi `app.seed-admin.enabled=true` và username chưa tồn tại. Mật khẩu băm bằng chính `PasswordEncoder` của ứng dụng, thay vì nhúng chuỗi băm

cứng trong file migration.

```
package com.fptaptech.catalog.auth;

import com.fptaptech.catalog.user.*;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import
org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

@Component
@ConditionalOnProperty(name = "app.seed-admin.enabled", havingValue = "true")
public class AdminSeeder implements ApplicationRunner {

    private static final Logger log =
LoggerFactory.getLogger(AdminSeeder.class);

    private final UserRepository users;
    private final RoleRepository roles;
    private final PasswordEncoder passwordEncoder;
    private final String username;
    private final String password;

    public AdminSeeder(UserRepository users, RoleRepository roles,
PasswordEncoder passwordEncoder,
                       @Value("${app.seed-admin.username}") String username,
                       @Value("${app.seed-admin.password}") String password)
{
        this.users = users;
        this.roles = roles;
        this.passwordEncoder = passwordEncoder;
        this.username = UserService.normalize(username);
        this.password = password;
    }

    @Override
    @Transactional
```

```
    public void run(ApplicationArguments args) {
        if (users.existsByUsername(username)) {
            return;
        }
        AppUser admin = new AppUser(username,
passwordEncoder.encode(password));
        admin.getRoles().add(roles.findByName(Role.USER).orElseThrow());
        admin.getRoles().add(roles.findByName(Role.ADMIN).orElseThrow());
        users.save(admin);
        log.info("Đã tạo tài khoản ADMIN '{}'", username);
    }
}
```

9. Tài liệu hóa bảo mật trên Swagger UI

Khai báo một security scheme kiểu bearer, rồi gắn nó vào đúng những operation cần token. Swagger UI sẽ hiện nút **Authorize** và biểu tượng ổ khóa cạnh các operation đó.

9.1 Khai báo scheme

Thêm annotation lên `OpenApiConfig` (bean `OpenAPI` từ tutorial REST API giữ nguyên):

```
import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
import io.swagger.v3.oas.annotations.security.SecurityScheme;

@Configuration
@SecurityScheme(
        name = "bearerAuth",
        type = SecuritySchemeType.HTTP,
        scheme = "bearer",
        bearerFormat = "JWT",
        description = "Lấy token từ POST /api/auth/token rồi dán vào đây
(không kèm chữ Bearer)")
public class OpenApiConfig {
    // bean OpenAPI giữ nguyên
}
```

9.2 Gắn scheme vào operation ghi

Trong `ProductApi`, thêm `@SecurityRequirement` và mô tả 2 mã lỗi bảo mật cho `create`, `update`, `delete`. Ví dụ với `create`:

```
import io.swagger.v3.oas.annotations.security.SecurityRequirement;

@Operation(summary = "Tạo sản phẩm mới")
@SecurityRequirement(name = "bearerAuth")
@ApiResponse(responseCode = "201", description = "Đã tạo")
@ApiResponse(responseCode = "400", description = "Dữ liệu không hợp lệ",
        content = @Content(mediaType = "application/problem+json",
                schema = @Schema(implementation = ProblemDetail.class)))
@ApiResponse(responseCode = "401", description = "Thiếu token, token hết hạn
hoặc sai chữ ký",
        content = @Content)
@ApiResponse(responseCode = "403", description = "Token hợp lệ nhưng không có
vai trò ADMIN",
        content = @Content)
@PostMapping
ResponseEntity<ProductResponse> create(@Valid @RequestBody ProductRequest
request);
```

`content = @Content` (rỗng) báo cho springdoc biết response 401/403 không có body JSON: Spring Security trả lỗi qua header `WWW-Authenticate`, ví dụ `Bearer error="invalid_token", error_description="Jwt expired at ..."`.

9.3 Dùng trên Swagger UI

1. Mở `http://localhost:8080/swagger-ui.html`, mục **Auth** → `POST /api/auth/token` → **Try it out** → gửi body mẫu → sao chép giá trị `accessToken`.
2. Bấm **Authorize** ở đầu trang, dán token vào ô `bearerAuth` → **Authorize** → **Close**.
3. Gọi `POST /api/v1/products`: Swagger UI tự gửi header `Authorization: Bearer <token>`. Nhờ `persist-authorization: true`, token vẫn còn sau khi tải lại trang.

10. Chạy và kiểm thử

10.1 Chạy và gọi bằng curl

```
./mvnw spring-boot:run
# Log: Successfully applied 2 migrations ... Đã tạo tài khoản ADMIN 'admin'

# 1. Lấy token
TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/token \
  -H 'Content-Type: application/json' \
```

```
  -d '{"username":"admin","password":"Admin#Catalog2026"}' | jq -r
.accessToken)

# 2. Xem thông tin token
curl -s http://localhost:8080/api/auth/me -H "Authorization: Bearer $TOKEN" |
jq

# 3. Tạo sản phẩm không có token -> 401
curl -i -X POST http://localhost:8080/api/v1/products \
  -H 'Content-Type: application/json' \
  -d '{"name":"Chuột không dây","price":350000}'

# 4. Tạo sản phẩm có token ADMIN -> 201
curl -i -X POST http://localhost:8080/api/v1/products \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"name":"Chuột không dây","price":350000}'
```

Tình huống	Kết quả mong đợi
`POST /api/auth/token` đúng mật khẩu	`200`, có `accessToken`, `expiresIn: 900`
`POST /api/auth/token` sai mật khẩu	`401`, body `application/problem+json`, `detail: "Sai username hoặc mật khẩu"`
`GET /api/v1/products` không token	`200`
`POST /api/v1/products` không token	`401`, header `WWW-Authenticate: Bearer`
`POST /api/v1/products` token chỉ có `ROLE_USER`	`403`
Bất kỳ request nào với token đã quá 15 phút	`401`, `WWW-Authenticate: Bearer error="invalid_token", error_description="Jwt expired at ..."`
Sửa 1 ký tự trong phần payload của token rồi gửi	`401`, chữ ký không còn khớp

10.2 Cấu hình test

Khóa bí mật thật không được commit, nên test dùng một cặp khóa riêng chỉ dành cho test, đặt trong `src/test/resources/keys/` (sinh bằng 2 lệnh `openssl` như mục 5.1, được phép commit). `src/test/resources/application-test.yaml`:

```
spring:
  datasource:
    url:
jdbc:h2:mem:catalogtest;MODE=MySQL;DATABASE_TO_LOWER=TRUE;DB_CLOSE_DELAY=-1
  docker:
    compose:
      enabled: false

app:
  jwt:
    public-key: classpath:keys/test.pub
    private-key: classpath:keys/test.key
  seed-admin:
    enabled: true
    username: admin
    password: Admin#Test2026
```

10.3 Test tự động

`jwt()` của `spring-security-test` tạo sẵn một `Authentication` dạng JWT cho request, không cần phát hành token thật — dùng để kiểm tra phân quyền. Một test riêng đi hết luồng thật: lấy token từ endpoint rồi dùng nó.

```
package com.fptaptech.catalog;

import com.fptaptech.catalog.product.ProductRequest;
import com.fptaptech.catalog.product.ProductService;
import com.jayway.jsonpath.JsonPath;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import
org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.jwt.Jwt;
```

```
import org.springframework.security.oauth2.jwt.JwtDecoder;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.test.web.servlet.MockMvc;

import java.math.BigDecimal;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.hamcrest.Matchers.startsWith;
import static org.springframework.http.HttpHeaders.AUTHORIZATION;
import static org.springframework.http.MediaType.APPLICATION_JSON;
import static
org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestP
ostProcessors.jwt;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class ApiSecurityTest {

    private static final String PRODUCT_JSON = """
            {"name":"Chuột không dây","description":"2.4GHz","price":350000}
            """;

    @Autowired MockMvc mvc;
    @Autowired JwtDecoder jwtDecoder;
    @Autowired ProductService productService;

    private String obtainToken(String username, String password) throws
Exception {
        String body =
mvc.perform(post("/api/auth/token").contentType(APPLICATION_JSON)
                        .content("
{\"username\":\"%s\",\"password\":\"%s\"}".formatted(username, password)))
                .andExpect(status().isOk())
                .andReturn().getResponse().getContentAsString();
```

```
        return JsonPath.read(body, "$.accessToken");
    }

    @Test
    void validCredentialsReturnSignedToken() throws Exception {
        Jwt jwt = jwtDecoder.decode(obtainToken("admin", "Admin#Test2026"));
        assertThat(jwt.getSubject()).isEqualTo("admin");
        assertThat(jwt.getClaimAsStringList("roles")).contains("ROLE_USER",
"ROLE_ADMIN");
    }

    @Test
    void wrongPasswordReturnsProblemDetail() throws Exception {
        mvc.perform(post("/api/auth/token").contentType(APPLICATION_JSON)
                        .content("
{\"username\":\"admin\",\"password\":\"wrong\"}"))
           .andExpect(status().isUnauthorized())
           .andExpect(jsonPath("$.title").value("Authentication failed"));
    }

    @Test
    void anyoneCanReadProducts() throws Exception {
        mvc.perform(get("/api/v1/products")).andExpect(status().isOk());
    }

    @Test
    void writeWithoutTokenReturns401WithBearerChallenge() throws Exception {

mvc.perform(post("/api/v1/products").contentType(APPLICATION_JSON).content(PR
ODUCT_JSON))
           .andExpect(status().isUnauthorized())
           .andExpect(header().string("WWW-Authenticate",
startsWith("Bearer")));
    }

    @Test
    void userRoleCannotCreateProduct() throws Exception {

mvc.perform(post("/api/v1/products").contentType(APPLICATION_JSON).content(PR
ODUCT_JSON)
                        .with(jwt().authorities(new
SimpleGrantedAuthority("ROLE_USER"))))
           .andExpect(status().isForbidden());
```

```
    }

    @Test
    void adminRoleCanCreateProduct() throws Exception {

mvc.perform(post("/api/v1/products").contentType(APPLICATION_JSON).content(PR
ODUCT_JSON)
                        .with(jwt().authorities(new
SimpleGrantedAuthority("ROLE_ADMIN"))))
           .andExpect(status().isCreated());
    }

    @Test
    void malformedTokenIsRejected() throws Exception {
        mvc.perform(get("/api/auth/me").header(AUTHORIZATION, "Bearer not-a-
real-token"))
           .andExpect(status().isUnauthorized());
    }

    @Test
    void realTokenWorksEndToEnd() throws Exception {
        String token = obtainToken("admin", "Admin#Test2026");
        mvc.perform(get("/api/auth/me").header(AUTHORIZATION, "Bearer " +
token))
           .andExpect(status().isOk())
           .andExpect(jsonPath("$.username").value("admin"));
    }

    @Test
    @WithMockUser(roles = "USER")
    void serviceRejectsNonAdminEvenWithoutHttp() {
        assertThatThrownBy(() -> productService.create(
                new ProductRequest("Bàn phím", null, BigDecimal.TEN)))
            .isInstanceOf(AccessDeniedException.class);
    }
}
```

Chạy: `./mvnw test`. Test cuối cùng chứng minh lớp bảo vệ thứ hai: gọi thẳng service, không qua HTTP, vẫn bị chặn.

11. Bài tập: Phiên đăng nhập an toàn cho Catalog API

Ứng dụng di động của cửa hàng cần giữ người dùng đăng nhập nhiều ngày, trong khi access token chỉ sống 15 phút. Đồng thời, đăng xuất phải có tác dụng thật, đổi mật khẩu phải làm mọi token cũ mất hiệu lực, và khóa ký phải thay được định kỳ mà không bắt mọi người đăng nhập lại.

Hoàn thành bài tập bằng cách mở rộng project của tutorial này để đáp ứng toàn bộ yêu cầu dưới đây.

11.1 Yêu cầu chức năng

Mã	Yêu cầu	Tiêu chí nghiệm thu
R1	Refresh token	`POST /api/auth/token` trả thêm `refreshToken`: chuỗi ngẫu nhiên 256 bit (không phải JWT), hạn 7 ngày; database chỉ lưu giá trị băm SHA-256 kèm `user_id`, `family_id`, `expires_at`, `revoked_at`, IP và user-agent lúc cấp
R2	Xoay vòng refresh token	`POST /api/auth/refresh` nhận refresh token, trả cặp access + refresh token mới; refresh token cũ bị đánh dấu đã dùng và không dùng lại được
R3	Phát hiện dùng lại	Gửi lại một refresh token đã dùng thì toàn bộ chuỗi token cùng `family_id` bị thu hồi, trả `401` với mã lỗi `REFRESH_TOKEN_REUSED`, ghi log mức WARN
R4	Đăng xuất	`POST /api/auth/logout` thu hồi refresh token hiện tại và đưa `jti` của access token đang dùng vào danh sách thu hồi cho đến khi token đó hết hạn
R5	Đăng xuất mọi thiết bị	`POST /api/auth/logout-all` thu hồi mọi refresh token của người dùng và làm mọi access token đã phát hành mất hiệu lực, dùng claim `ver` so với cột `token_version` của bảng `users`
R6	Thu hồi khi thay đổi tài khoản	Đổi mật khẩu (`POST /api/auth/change-password`) hoặc ADMIN khóa tài khoản đều tăng `token_version`

Mã	Yêu cầu	Tiêu chí nghiệm thu
R7	Xoay khóa ký	Hỗ trợ ít nhất 2 khóa RSA cùng lúc, phân biệt bằng `kid`; token mới ký bằng khóa đang "active"; token ký bằng khóa cũ vẫn hợp lệ đến khi hết hạn; đổi khóa active chỉ bằng cấu hình, không sửa code
R8	Công bố khóa công khai	`GET /.well-known/jwks.json` trả mọi khóa công khai đang hiệu lực theo định dạng JWK Set; tuyệt đối không lộ phần khóa bí mật
R9	Phân quyền theo scope	Token có thêm claim `scope` sinh từ vai trò: USER → `product:read`, ADMIN → `product:read product:write`; thao tác ghi sản phẩm yêu cầu `SCOPE_product:write` ở cả quy tắc URL và `@PreAuthorize`
R10	Giới hạn tần suất	`/api/auth/token` và `/api/auth/refresh`: tối đa 10 request mỗi phút mỗi IP; vượt thì trả `429` kèm header `Retry-After` và body ProblemDetail
R11	Tài liệu API	Mọi endpoint mới có trên Swagger UI với mô tả đầy đủ request, response và từng mã lỗi (`400`, `401`, `403`, `429`)

11.2 Yêu cầu phi chức năng

- Thu hồi (R4–R6) có hiệu lực chậm nhất 5 giây sau khi thực hiện. Được phép cache trạng thái thu hồi, nhưng không truy vấn database quá 1 lần cho mỗi request.
- Việc kiểm tra thu hồi và `token_version` nằm trong khâu kiểm tra token của `JwtDecoder`, không viết filter riêng chen vào chuỗi filter.
- Bản ghi trong danh sách thu hồi tự hết hạn cùng access token; refresh token hết hạn được dọn bằng job định kỳ mỗi ngày.
- Không ghi access token, refresh token hay mật khẩu ra log ở bất kỳ mức nào.
- So sánh giá trị băm của refresh token bằng phép so sánh thời gian hằng (constant-time).

11.3 Ràng buộc kỹ thuật

Spring Boot 4.1.x, JDK 25, Spring Security 7.1 (OAuth2 Resource Server), MySQL 8.4, Flyway. Mọi thay đổi schema là migration mới (`V3__...` , `V4__...`); không sửa `V1` , `V2` . Không dùng authorization server bên ngoài (Keycloak, Auth0…): ứng dụng tự phát hành token như trong tutorial.

11.4 Sản phẩm nộp

- Mã nguồn hoàn chỉnh, chạy được bằng `./mvnw spring-boot:run` , kèm `README` mô tả cách cấu hình và xoay khóa.
- Bộ test tự động chứng minh từng yêu cầu, tối thiểu gồm: xoay vòng refresh token thành công; dùng lại refresh token cũ làm cả chuỗi bị thu hồi; access token bị từ chối sau logout; mọi token bị từ chối sau logout-all và sau đổi mật khẩu; token ký bằng khóa cũ vẫn được chấp nhận; token có `kid` lạ bị từ chối; request thứ 11 trong 1 phút nhận `429` .
- Sơ đồ trình tự (sequence diagram) cho 3 luồng: cấp token lần đầu, refresh có xoay vòng, phát hiện dùng lại refresh token.
- File `docs/api/openapi.yaml` xuất từ springdoc sau khi hoàn thành.